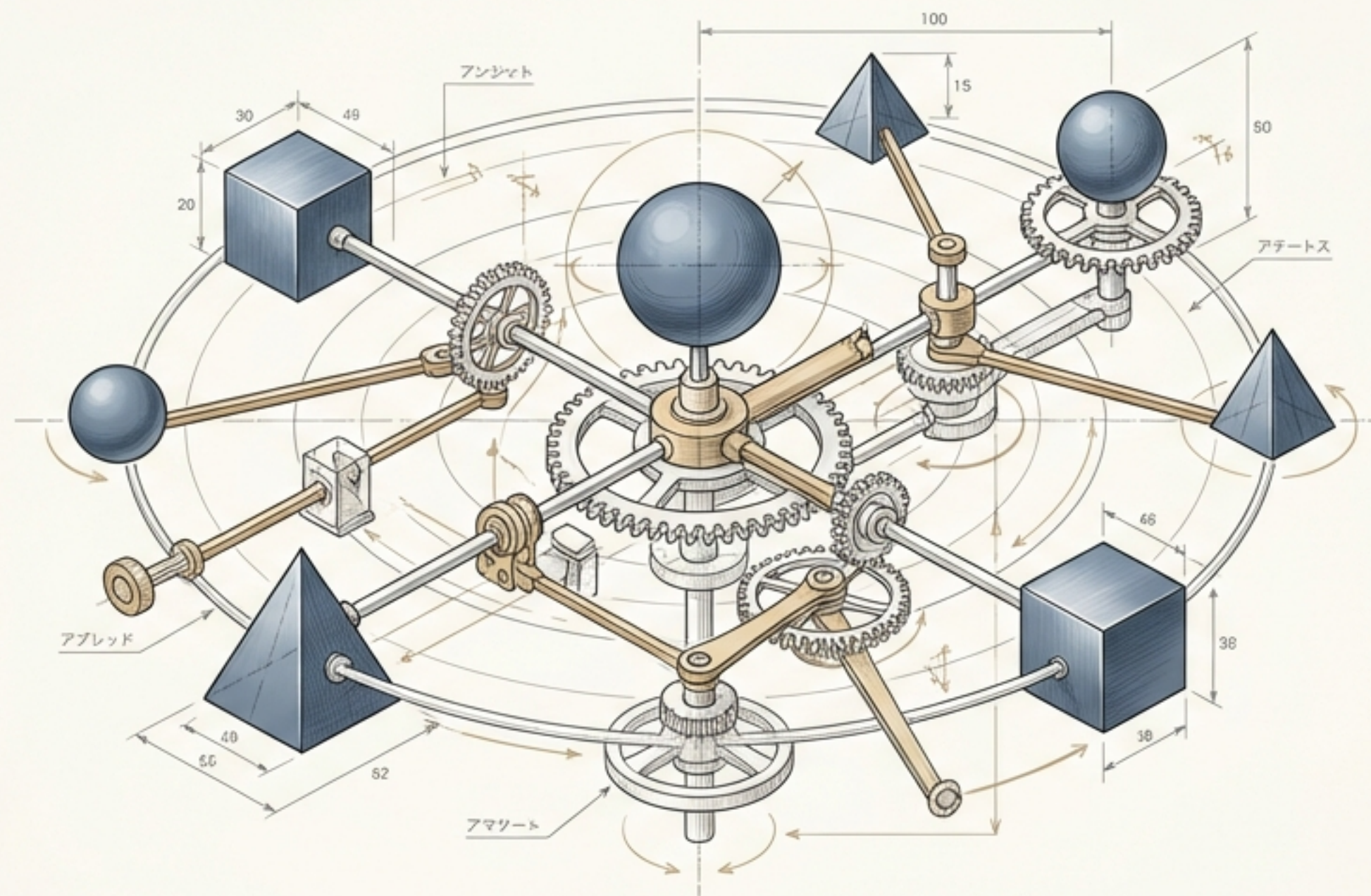
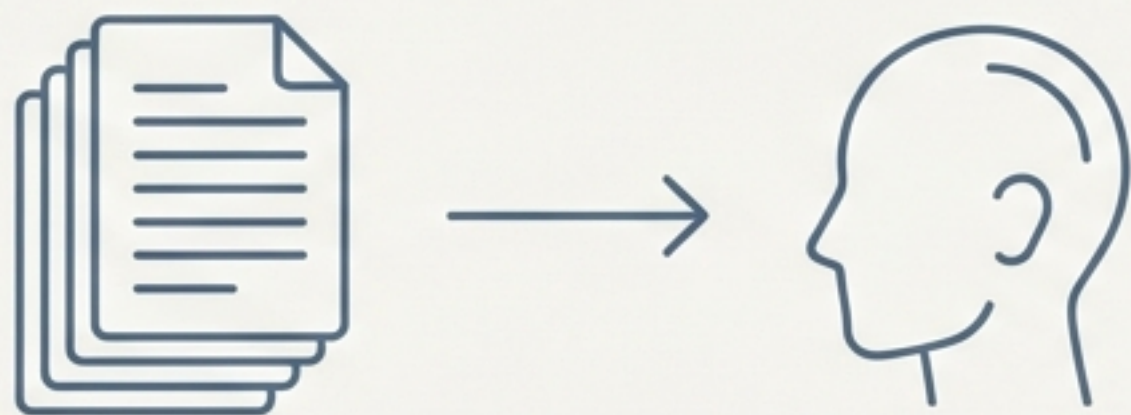




AI開発の次なるフロンティア：AutoGenで実現する 自律型マルチエージェントシステム構築

開発者のための、AutoGenフレームワーク入門

生成AIのトレンドは「回答」から「実行」へ



「RAG: 回答するAI」

Key Message 1: 2025年、AI開発の主戦場はRAG (Retrieval Augmented Generation) から「AI エージェント」へと大きくシフトしています。

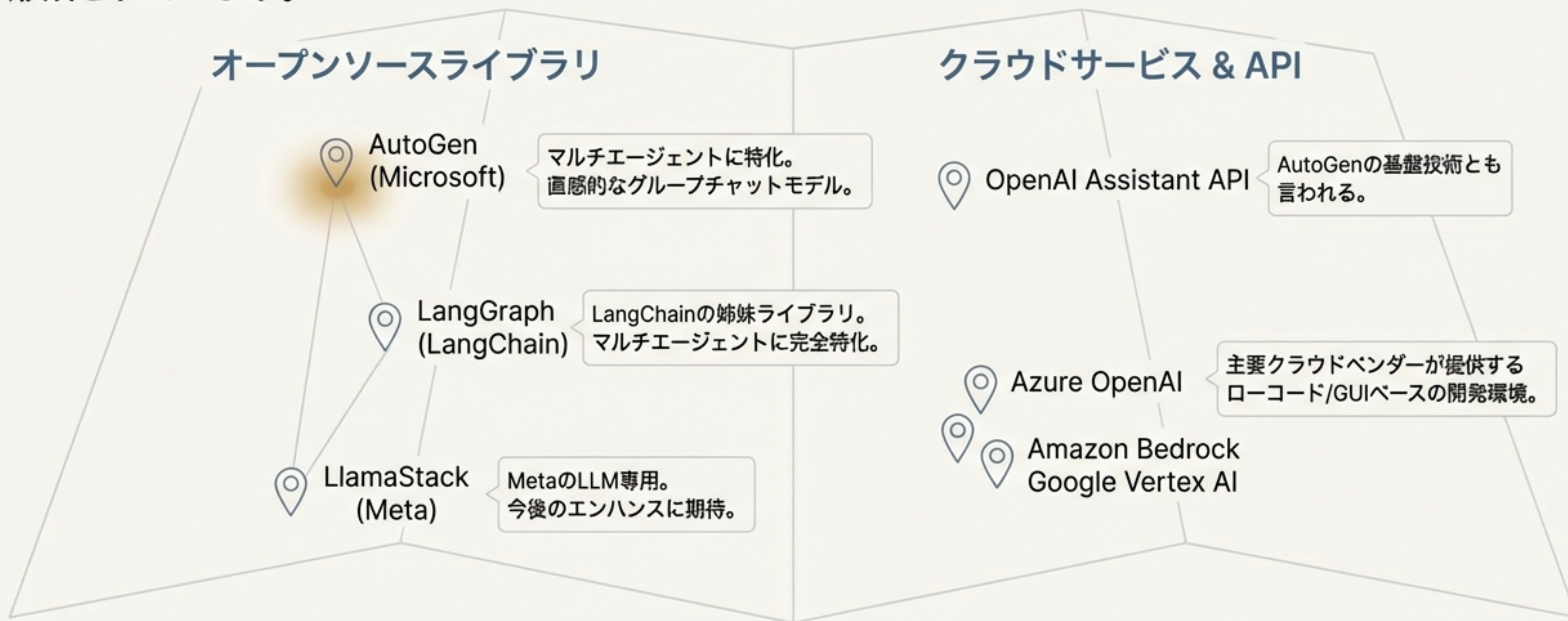


「AIエージェント: 働かせるAI」

Key Message 2: これまでのLLMが担ってきた役割は「質問に回答する」ことでした。しかし、AIエージェントはLLMに「自律的に働かせる」ことを目指します。

AIエージェント開発を巡る広大なエコシステム

マルチエージェントシステムの構築は複雑ですが、開発を支援する多様なツールが登場し、エコシステムが形成されています。



Takeaway: 数ある選択肢の中で、本セッションでは最もハードルが低く、かつ強力な「AutoGen」に焦点を当てます。

なぜAutoGenなのか？ その答えは「グループチャット」という発想

Key Concept: AutoGenの最大の特徴は、私たちが普段使い慣れているグループチャットの仕組みを、そのまま プログラミングモデルとして採用している点にあります。



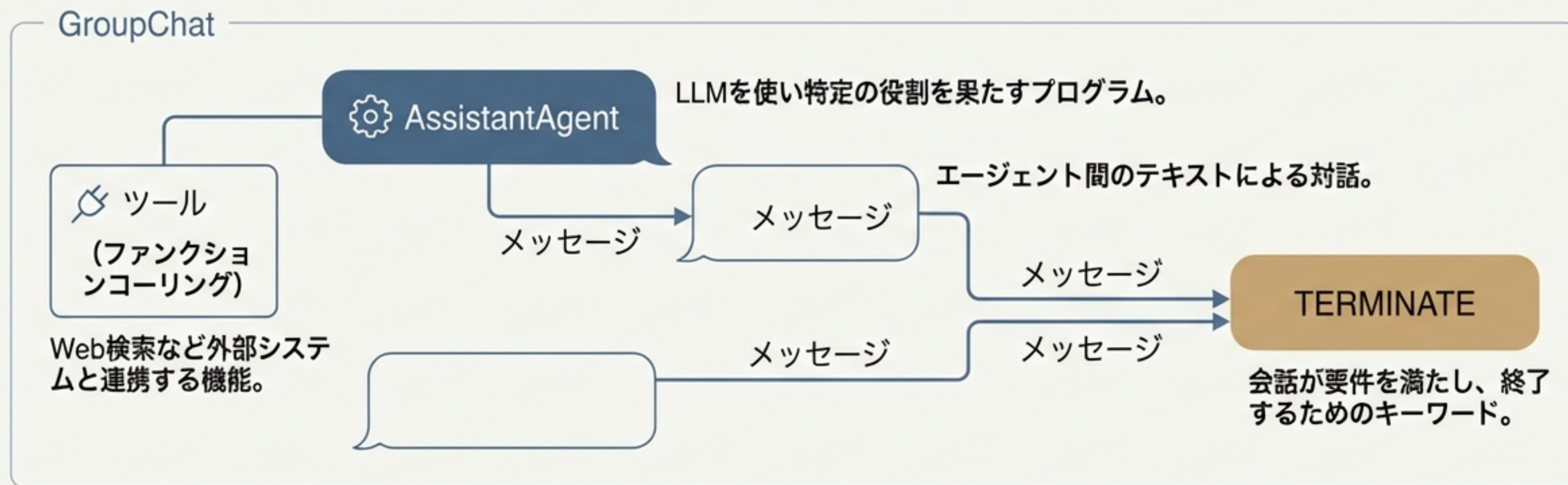
- * **直感的な開発体験:** LINEやSlackでの会話のように、エージェント間の連携を設計できる。
- * **エコシステムのハブ:** LangChainやDockerなど、既存のデファクトスタンダードツールと強力に連携可能。
- * **明確なポジショニング:** MicrosoftのSemantic Kernelが汎用アプリ用であるのに対し、AutoGenはマルチエージェントに特化しており、役割分担が明確。

AutoGenのアーキテクチャ：3つの主要コンポーネント



AgentChatプログラミングモデル：会話でシステムを構築する

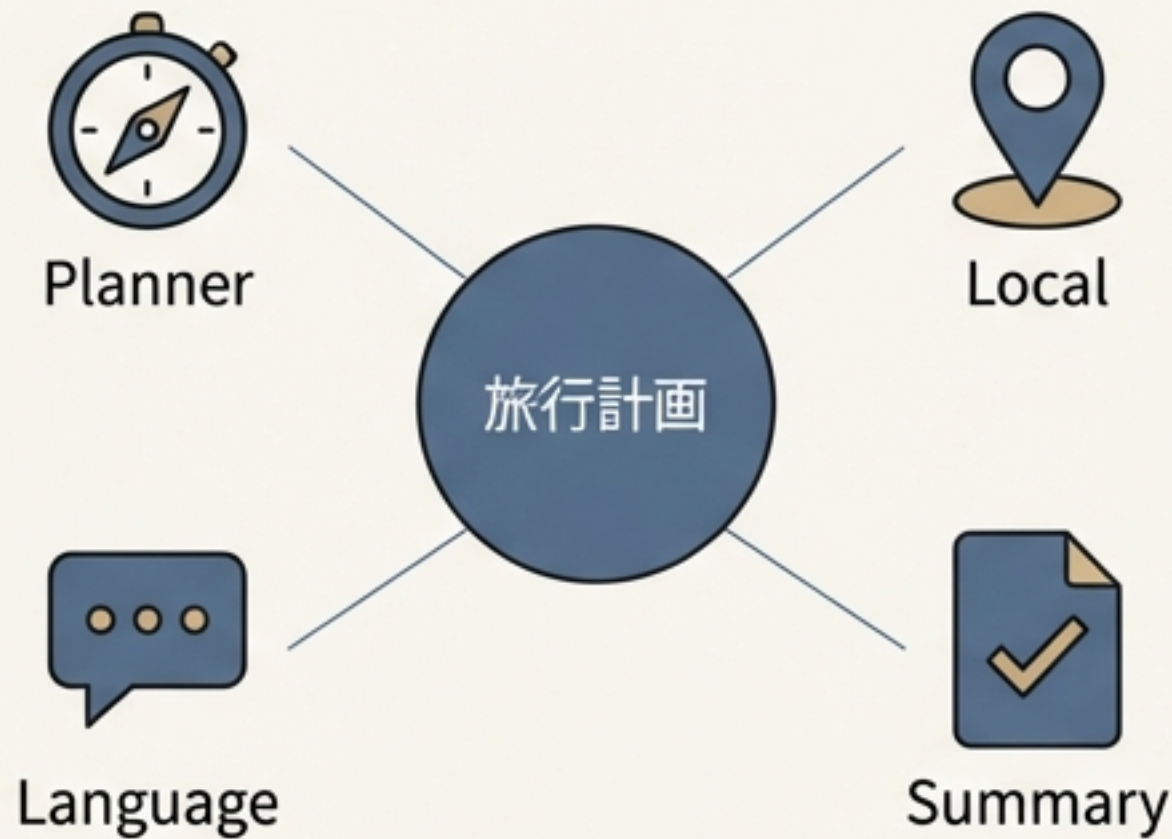
「LINEやSlackのような挙動をAPIとして表現したライブラリ」



Key Takeaway: この直感的なモデルにより、高い抽象度と少ないコード量での開発が実現します。

実践パターン①：役割分担による協調型タスク解決

Use Case: 複数の専門家エージェント（旅行プランナー、現地情報担当など）が協力し、一つの旅行計画を完成させる。



Takeaway: 各エージェントに明確な役割（`system_message`）を与えることで、自律的な協調作業が生まれる。

Key Code Snippet

```
# エージェントの役割はSystem Messageで定義する
planner_agent = autogen.AssistantAgent(
    name="PlannerAgent",
    system_message="""あなたはユーザーのリクエストに基づき旅行提案できるアシスタントです。
    具体的な旅行計画（日程、目的地、アクティビティなど）を立案してください。"""
)

travel_summary_agent = autogen.AssistantAgent(
    name="TravelSummaryAgent",
    system_message="""あなたはこれまでのエージェントの出力を取りまとめて、1つの旅行予定にまとめるアシスタントです。
    最終的な計画が完成したら、TERMINATEで応答できます。"""
)
```


サンプルコード全体像：役割分担型マルチエージェント

以下は、ネパール旅行を計画する4つのエージェントを定義し、実行するコードの全体像です。

```
import autogen

# LLM設定
config_list = autogen.config_list_from_json(...)

# エージェント定義 (Planner, Local, Language, Summary)
planner_agent = autogen.AssistantAgent(name="PlannerAgent", ...)
local_agent = autogen.AssistantAgent(name="LocalAgent", ...)
language_agent = autogen.AssistantAgent(name="LanguageAgent", ...)
travel_summary_agent = autogen.AssistantAgent(name="TravelSummaryAgent", ...)

# グループチャットとマネージャーの定義
group_chat = autogen.GroupChat(
    agents=[planner_agent, local_agent, language_agent, travel_summary_agent],
    messages=[],
    max_turns=10,
    speaker_selection_method="round_robin",
)
manager = autogen.GroupChatManager(groupchat=group_chat, ...)

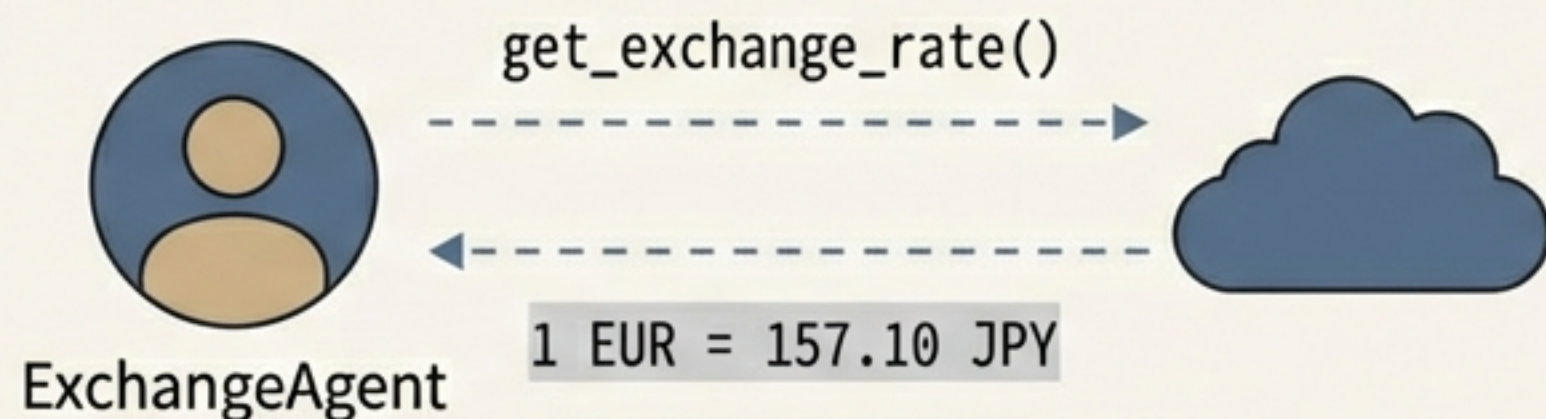
# ユーザープロキシ (チャットの起点)
user_proxy = autogen.UserProxyAgent(
    name="User",
    human_input_mode="NEVER",
    is_termination_msg=lambda x: "TERMINATE" in x.get("content", ""),
)

# チャット開始
user_proxy.initiate_chat(
    manager,
    message="ネパールへの3日間の旅行を計画してください。",
)
```

Note: このコードは、後で共有する資料からコピーしてすぐに試すことができます。

実践パターン②：ファンクションコーリングによる機能拡張

Use Case: 旅行計画エージェントが、外部APIを呼び出してリアルタイムの為替レートを取得する。



Takeaway: Python関数を定義し`tools`に渡すだけで、エージェントは自律的に外部情報を取得・活用できるようになる。

Key Code Snippet

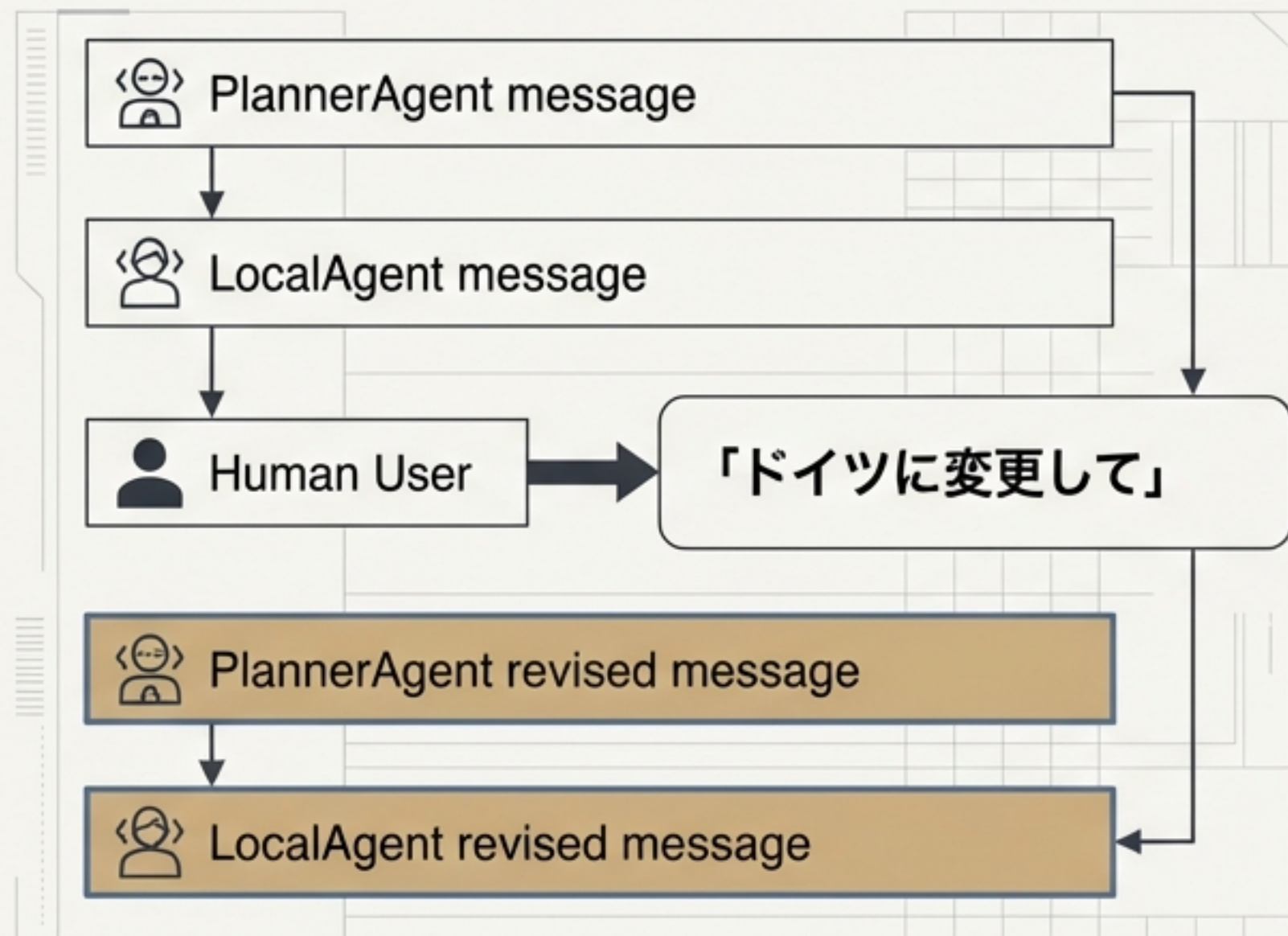
```
# 1. Python関数を定義する
def get_exchange_rate(currency_code: str) -> str:
    # ... yfinanceライブラリで為替レートを取得 ...
    return f"1 {currency_code} = {latest_rate:.2f} JPY"

# 2. 関数をツールとしてラップする
exchange_rate_tool = autogen.FunctionTool(
    func=get_exchange_rate, ...
)

# 3. エージェントにツールを関連付ける
exchange_agent = autogen.AssistantAgent(
    name="ExchangeAgent",
    system_message="get_exchange_rate_toolを使って為替レートを提供してください。",
    tools=[exchange_rate_tool],
)
```


実践パターン③：ユーザー介入によるプロセスの制御

Use Case: AIが生成した旅行計画を人間がレビューし、修正を指示したり、最終承認を与えたりする。



Key Code Snippet

```
# UserProxyAgentの設定を変更する
user_proxy = autogen.UserProxyAgent(
    name="User",
    # 毎ターン、人間の入力を必須にする
    human_input_mode="ALWAYS",
    # ユーザーが "approve" と入力したらチャットを終了する
    is_termination_msg=lambda x: x.get("content",
        "").rstrip().endswith("approve"),
)

# グループチャットにuser_proxyを追加する
group_chat = autogen.GroupChat(
    agents=[user_proxy, planner_agent, ...], ...
)
```

Takeaway: `human_input_mode="ALWAYS"`を設定するだけで、自律的なエージェントの会話に人間が介入し、承認や指示出しが可能になる。

グループチャットに設定を変更する

実践パターン④：交渉型マルチエージェントによる合意形成

Use Case: 不動産の「売り手」「買い手」「仲介者」エージェントが、それぞれの目標（高く売る/安く買う/合意形成）に基づき価格交渉を行う。



Takeaway: `system_message`を工夫することで、単なる協力関係だけでなく、利害が対立するエージェント間の交渉シミュレーションも構築できる。

Key Code Snippet

```
# 各エージェントに相反する目標を与える
seller_agent = autogen.AssistantAgent(
    name="SellerAgent",
    system_message="あなたは売り手です。できるだけ高い価格で売却したい...",
)

buyer_agent = autogen.AssistantAgent(
    name="BuyerAgent",
    system_message="あなたは買い手です。できるだけ低い価格で購入したい...",
    tools=[real_estate_search_tool], # 買い手は市場価格を調査
)

negotiation_agent = autogen.AssistantAgent(
    name="NegotiationAgent",
    name="NegotiationAgent",
    system_message="あなたは仲介者です。双方にとって公平な価格を見つけてください...",
)
```


交渉の実行結果：エージェントたちの対話シミュレーション

User: 品川駅徒歩10分、70平米のマンションを売却したい。

SellerAgent: 素晴らしい物件ですね！希望価格は1億2000万円です。

BuyerAgent: (Calling tool "search_real_estate"...)

市場価格を調査したところ、1億1000万円が相場ようです。

SellerAgent: もう少し価値があると考えております。1億1500万円ではいかがでしょうか。

BuyerAgent: 1億1200万円が限界です。

NegotiationAgent: 間を取り、1億1350万円ではいかがでしょうか。

SellerAgent: 検討の余地があります。

BuyerAgent: 合意いたします。

NegotiationAgent: 交渉成立！

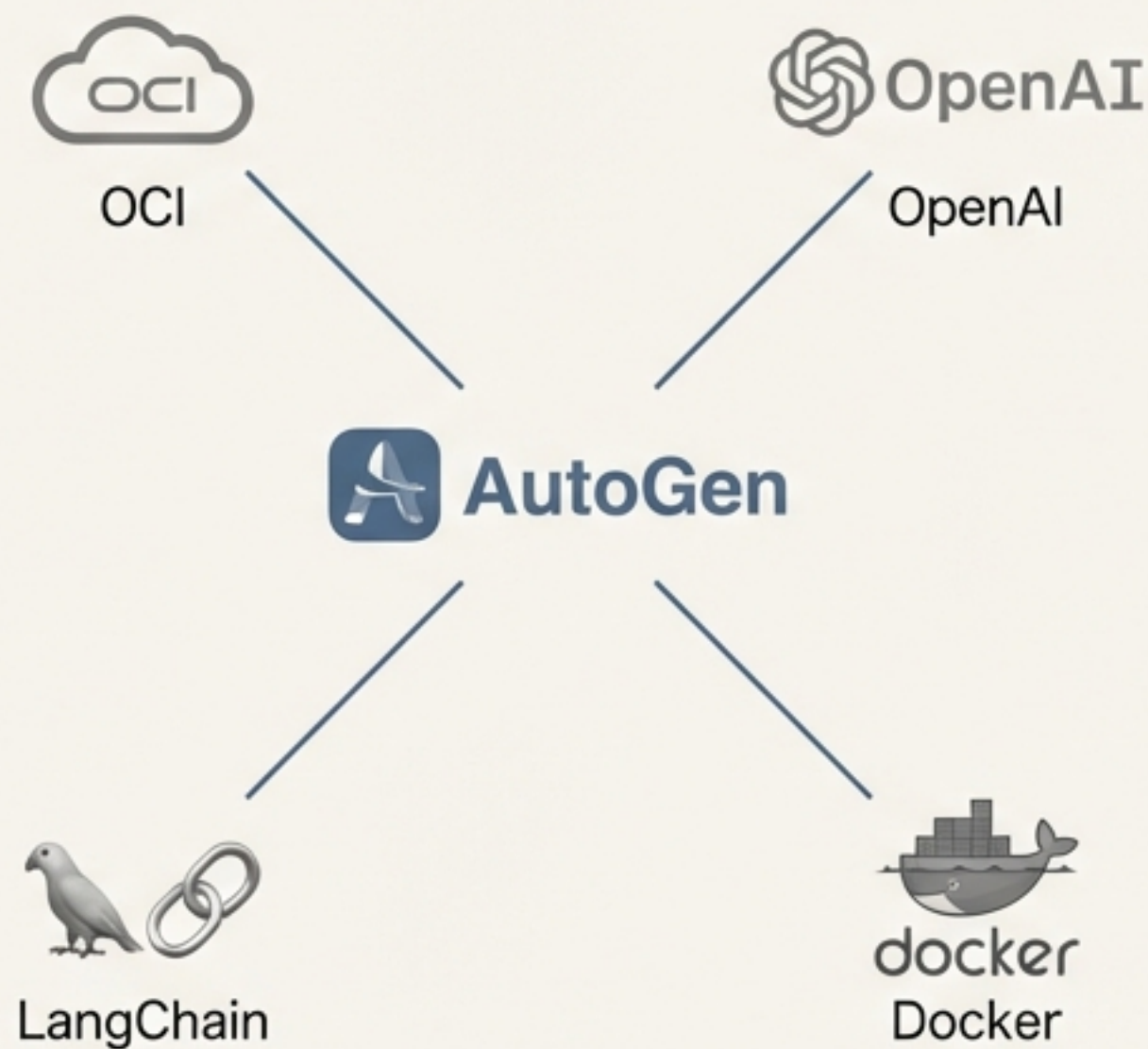
Takeaway: 各エージェントが自身の役割と目標に基づき、ファンクションコーリングで得た情報を活用しながら、自律的に交渉を進めている。

高度な活用：カスタムLLMとエコシステム連携

Key Message: AutoGenはOpenAIに限定されません。柔軟な設計により、様々なLLMやツールと連携できます。

Content Block 1: カスタムLLMの利用

- **Example:** Oracle Cloud Infrastructure (OCI) の Generative AI サービス (Cohere モデル) を利用する。
- **How:** `OAI\_CONFIG\_LIST` ファイルにOCIのエンドポイントやモデル情報を記述するだけで、エージェントが使用するLLMを切り替え可能。
- **Benefit:** 特定の要件やコスト、セキュリティポリシーに合わせて最適なLLMを選択できる。



Content Block 2: 既存エコシステムとの連携

- **LangChain:** LangChainで定義されたエージェントやツールとAutoGenを連携させ、広範なエコシステムを活用できる。
- **Docker:** コード実行環境としてDockerコンテナを指定でき、安全で再現性の高いエージェント実行が可能。

AutoGenアドバンテージ：マルチエージェント開発の最適な選択肢



1. 直感的なグループチャットモデル

学習コストが低く、エージェント間の複雑な連携を直感的に設計できる。



2. 高い抽象度と生産性

少ないステップ数のコードで高度なエージェントシステムを記述でき、開発効率が大幅に向上する。



3. 豊富なリソースとコミュニティ

充実した公式ドキュメントと活発なコミュニティにより、学習や問題解決が容易。



4. 圧倒的な柔軟性と連携性

カスタムLLMをサポートし、LangChainなどの既存ツールとシームレスに連携できる。

自律型エージェントの時代が、今始まる

これからのAIアプリケーション開発において、
マルチエージェントの概念は中心的な役割を果たします。

AutoGenは、その複雑な世界を航海するための、
最も信頼できる地図とツールを提供します。

公式ドキュメントと豊富なサンプルコードを手に、
今日からあなたのマルチエージェント開発を始めましょう。